

Purpose

This document defines the technical design of each integration flow between Kommo CRM and Aloware.

Unlike the Architecture Design document, which describes the solution at a high level, this document focuses on how individual synchronization processes are structured and how they interact with the participating systems.

The objective is to establish a consistent design for every integration flow before implementation.

Scope

This document describes:

- Integration flows
- Trigger mechanisms
- Participating systems
- Data direction
- High-level processing sequence
- Expected outputs
- Design considerations

It intentionally excludes implementation details such as API endpoints, field mappings, validation rules, retries, logging, and Make module configuration.

Flow Design Principles

Every integration flow should follow the same architectural principles.

Independent Execution

Each flow operates independently from all others.

A failure in one flow must not interrupt unrelated synchronizations.

Event-Driven Processing

Flows are initiated by business events whenever webhook support is available.

Polling should only be used when no event mechanism exists.

Stateless Processing

Every execution should contain all information required to complete the synchronization.

No workflow should depend on execution state stored within Make.

Deterministic Processing

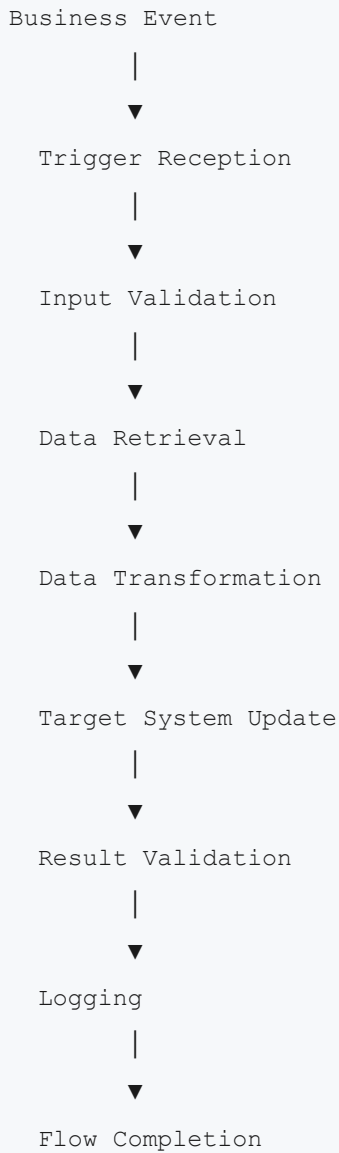
Given the same input, a flow should always produce the same expected result.

Idempotent Operations

Whenever possible, flows should safely support repeated execution without creating duplicate business effects.

Standard Flow Lifecycle

Every integration flow follows the same logical sequence.



This lifecycle is shared by every synchronization process.

Integration Flow Catalog

Flow 01 — Kommo → Aloware

Objective

Synchronize commercial events from Kommo into Aloware so communication campaigns always operate with current CRM information.

Source System

Kommo CRM

Target System

Aloware

Trigger

Business events generated in Kommo.

Examples include:

- Lead stage changes
- Lead creation
- Lead updates
- Contact updates
- Owner assignment

High-Level Process

1. Receive business event.
2. Validate payload.
3. Retrieve additional CRM information if required.
4. Transform business data.
5. Send data to Aloware.
6. Validate response.
7. Record execution outcome.

Expected Result

Aloware reflects the latest commercial information required for outbound communication.

Flow 02 — Aloware → Kommo

Objective

Synchronize communication events generated in Aloware back into Kommo.

Source System

Aloware

Target System

Kommo CRM

Trigger

Communication events generated in Aloware.

Examples include:

- Call completed
- SMS sent
- SMS received
- AI Summary generated
- Call recording available

High-Level Process

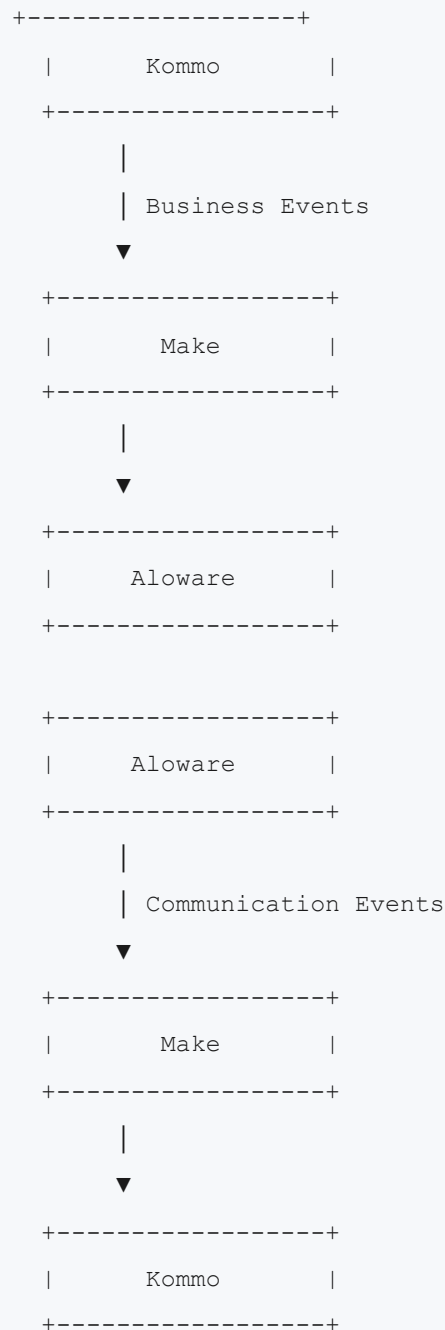
1. Receive communication event.
2. Validate payload.
3. Retrieve additional communication information if necessary.
4. Transform communication data.
5. Update Kommo.
6. Validate update.
7. Record execution outcome.

Expected Result

Kommo contains an accurate communication history associated with each lead.

Flow Interaction Model

Although both synchronization directions are independent, they form a complementary integration.



The two synchronization directions remain logically separated while sharing the same orchestration platform.

Common Processing Stages

Every flow consists of the following conceptual stages.

Stage	Purpose
Trigger Reception	Receive business event
Validation	Verify event integrity
Data Retrieval	Obtain required information
Transformation	Convert source data into target format
Synchronization	Execute target operation
Verification	Confirm successful execution
Logging	Register execution details
Completion	Finish workflow

Flow Boundaries

Each flow owns its complete execution lifecycle.

Responsibilities include:

- Receiving events
- Processing data
- Updating the target system
- Recording operational information

Responsibilities explicitly excluded from individual flows include:

- Global monitoring
- Retry policies
- Security enforcement
- Authentication management
- Cross-flow coordination

These concerns are addressed by dedicated design documents.

Scalability

The architecture allows additional flows to be incorporated without modifying existing ones.

Examples include:

- Contact synchronization
- Company synchronization

- Appointment synchronization
- AI event synchronization
- Marketing automation
- Reporting integrations

Each new flow should follow the same lifecycle and design principles defined in this document.

Design Assumptions

The flow design assumes:

- Reliable webhook delivery when supported.
 - REST APIs as the communication mechanism.
 - Platform-specific authentication.
 - Independent execution of every synchronization.
 - Eventual consistency between systems.
-

Related Documents

This document should be read together with:

- Architecture Design
- Synchronization Model
- Field Mapping
- Stage Mapping
- Agent Mapping
- Data Transformation Rules
- Validation Rules
- Error Handling Strategy
- Retry Strategy
- Logging Strategy
- Monitoring Strategy
- Security Considerations

Description

TBD

Documento	Descripción
Architecture Design	Define la arquitectura técnica de alto nivel de la integración, los componentes participantes, los patrones arquitectónicos adoptados y los principios que gobiernan la solución. Constituye la base sobre la cual se diseñan todos los demás documentos técnicos.
Integration Flows Design	Describe la estructura técnica de cada flujo de integración entre Kommo y Aloware, definiendo su objetivo, dirección, secuencia de procesamiento y responsabilidades, sin entrar aún en detalles de implementación.
Synchronization Model	Especifica cómo se sincronizan las entidades entre los sistemas, indicando la dirección de la sincronización, el sistema propietario de cada dato, la frecuencia, los disparadores y el modelo de consistencia utilizado.
Field Mapping	Define la correspondencia entre los campos de Kommo y Aloware, indicando qué información se sincroniza, cómo se representa en cada plataforma y las reglas generales de equivalencia.
Stage Mapping	Documenta la relación entre las etapas del pipeline de Kommo y los estados o listas equivalentes en Aloware, estableciendo las reglas de sincronización entre ambos modelos de negocio.
Agent Mapping	Define la equivalencia entre usuarios, agentes o propietarios existentes en ambas plataformas para garantizar la correcta asignación de responsables durante la sincronización.
Data Transformation Rules	Especifica las reglas de transformación necesarias para adaptar los datos entre ambos sistemas, incluyendo conversiones de formato, normalización, enriquecimiento y estandarización de valores.
Validation Rules	Describe las validaciones técnicas que deben realizarse antes de ejecutar una sincronización, asegurando la integridad, consistencia y calidad de los datos intercambiados.
Error Handling Strategy	Define cómo deben detectarse, clasificarse y gestionarse los errores producidos durante la integración, estableciendo criterios para su tratamiento y recuperación.
Retry Strategy	Especifica la política de reintentos ante errores temporales, definiendo cuándo volver a ejecutar una operación, bajo qué condiciones y cuándo considerar una falla definitiva.
Logging Strategy	Establece la información que debe registrarse durante cada ejecución de la integración para facilitar auditorías, diagnóstico de problemas y trazabilidad operativa.
Monitoring Strategy	Define cómo se supervisará el estado de la integración mediante métricas, alertas e indicadores operativos que permitan detectar incidentes y evaluar el funcionamiento de la solución.
Security Considerations	Documenta las consideraciones de seguridad aplicables a la integración, incluyendo autenticación, autorización, protección de credenciales, tratamiento de datos y buenas prácticas de comunicación entre sistemas.
Technical Design Summary	Resume las principales decisiones adoptadas durante la fase de diseño técnico, consolidando la arquitectura, los modelos de sincronización y las estrategias definidas antes de iniciar la implementación.

1. Technical Design/

| |— 01. Architecture Design.md |— 02. Integration Flows Design.md |— 03. Synchronization Model.md |— 04. Field Mapping.md |— 05. Stage Mapping.md |— 06. Agent Mapping.md |— 07. Data Transformation Rules.md |— 08. Validation Rules.md |— 09. Error Handling Strategy.md |— 10. Retry Strategy.md |— 11. Logging Strategy.md |— 12. Monitoring Strategy.md |— 13. Security Considerations.md |— 14. Technical Design Summary.md | |— flows/

```
|— F01 - Lead Stage Synchronization.md
  |— F02 - Call Activity Synchronization.md
  |— F03 - SMS Synchronization.md
  |— F04 - AI Summary Synchronization.md
  |— ...
```

Purpose

This document defines the high-level technical architecture of the Kommo ↔ Aloware integration.

Its purpose is to describe how the participating systems interact, the architectural principles that guide the solution, and the responsibilities of each component before detailing individual synchronization flows or implementation specifics.

This document intentionally avoids implementation details such as API requests, field mappings, Make scenarios, or retry logic, which are documented separately.

Design Objectives

The architecture has been designed to achieve the following objectives:

- Maintain a clear separation of responsibilities between platforms.
 - Synchronize business data with minimal manual intervention.
 - Minimize coupling between systems.
 - Support independent evolution of each platform.
 - Ensure scalability for future automation flows.
 - Improve resiliency through event-driven processing.
 - Preserve data ownership across systems.
 - Facilitate monitoring, troubleshooting, and maintenance.
-

Architectural Style

The solution follows a combination of architectural patterns.

Event-Driven Architecture

Business events generated by either platform initiate synchronization processes.

Examples include:

- Lead stage changes
- Call completion
- SMS events
- Contact updates
- AI Summary generation

The architecture reacts to events instead of relying on continuous polling whenever webhook support is available.

Orchestration Pattern

Make acts as the orchestration layer between systems.

Rather than allowing direct communication between Kommo and Aloware, every synchronization passes through Make, where business logic is centralized.

This approach provides:

- Centralized workflow management
 - Data transformation
 - Validation
 - Error handling
 - Logging
 - Monitoring
 - Future extensibility
-

API-Based Integration

Data exchange between systems occurs through REST APIs.

Communication is stateless and request-based.

Each synchronization is executed independently using authenticated API requests.

Loose Coupling

Neither Kommo nor Aloware contains business logic specific to the other platform.

Platform-specific behavior remains isolated inside the integration layer.

This reduces dependencies and simplifies future platform changes.

System Components

Kommo CRM

Primary responsibilities:

- Customer relationship management
- Lead lifecycle management
- Sales pipeline
- Contacts
- Companies
- Sales ownership

Kommo is the authoritative source for commercial information.

Aloware

Primary responsibilities:

- Outbound calling
- Inbound calling
- SMS
- Call recordings
- AI Call Summary
- Communication history

Aloware is the authoritative source for communication data.

Make

Primary responsibilities:

- Workflow orchestration
- Event processing
- API communication
- Data transformation

- Validation
- Error management
- Retry execution
- Logging
- Monitoring integration processes

No business data is permanently owned by Make.

High-Level Interaction Model

The integration is organized around independent synchronization flows.

Each flow follows the same logical lifecycle:

1. Business event occurs.
2. Event is received by Make.
3. Event is validated.
4. Required data is retrieved.
5. Data is transformed.
6. Target API is executed.
7. Result is validated.
8. Logs are generated.
9. Process finishes.

Each synchronization is independent from the others.

Communication Model

The solution combines two communication mechanisms.

Event Reception

Preferred mechanism:

- Webhooks

Alternative mechanism:

- Scheduled polling (only if required by platform limitations)
-

Data Exchange

Communication between systems is performed through authenticated REST APIs.

No direct database access exists between platforms.

Synchronization Principles

The architecture follows these synchronization principles.

System of Record

Each business entity has a single authoritative owner.

Commercial information belongs to Kommo.

Communication information belongs to Aloware.

Ownership is never shared.

Eventual Consistency

The systems are not expected to remain synchronized in real time.

Synchronization occurs after relevant events are processed successfully.

Short synchronization delays are considered acceptable.

Independent Processing

Each synchronization executes independently.

A failure in one workflow must not interrupt unrelated synchronization processes.

Stateless Execution

Each workflow execution contains all information required to complete its operation.

No execution depends on previous runtime state stored within Make.

Idempotent Design

Whenever possible, synchronization operations should be repeatable without producing duplicate business effects.

This minimizes the impact of retries and duplicate event delivery.

Architectural Principles

The technical design is guided by the following principles.

Separation of Responsibilities

Each platform performs only the functions that belong to its domain.

Single Source of Truth

Each data domain has a single owner.

Explicit Data Flow

Every data movement between systems should be intentional, documented, and traceable.

Centralized Integration Logic

Business rules specific to synchronization remain inside the integration layer rather than inside either platform.

Failure Isolation

Errors should affect only the current synchronization process.

Other workflows must continue operating normally.

Observability

Every synchronization should generate sufficient operational information to support diagnostics and auditing.

Extensibility

The architecture should allow future synchronization flows to be incorporated without requiring structural redesign.

Architectural Constraints

The design assumes the following constraints.

- Communication occurs exclusively through supported APIs and Webhooks.
 - No direct database integration exists.
 - Platform permissions define accessible operations.
 - API rate limits must be respected.
 - Authentication mechanisms remain platform-specific.
 - Make serves as the only orchestration layer.
-

Out of Scope

The following topics are intentionally excluded from this document:

- Detailed Make scenario design
- Field mappings
- Pipeline mappings
- Agent mappings
- API endpoint specifications
- Retry implementation
- Error handling procedures
- Logging implementation
- Monitoring implementation
- Security implementation details

These topics are documented independently within the Technical Design phase.

Related Documents

This document is complemented by:

- Integration Flows Design
- Synchronization Model
- Field Mapping
- Stage Mapping
- Agent Mapping
- Data Transformation Rules
- Validation Rules
- Error Handling Strategy
- Retry Strategy
- Logging Strategy
- Monitoring Strategy
- Security Considerations
- Technical Design Summary